Task-20:- Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to define and analyze the requirements for implementing and comparing two DNS management approaches:

- Self-hosted DNS server on AWS EC2
- Managed DNS using AWS Route 53

This document is intended for DevOps engineers, cloud engineers, system administrators, and stakeholders who want to understand DNS resolution behavior and infrastructure differences.

1.2 Scope

Included:

- Setup of a self-hosted DNS server using BIND9/CoreDNS on EC2
- Creation of DNS records (A, CNAME)
- Setup of AWS Route 53 hosted zone
- NS record switching at domain registrar
- DNS testing using dig and nslookup

Not Included:

- Domain registration process
- DNSSEC implementation
- Advanced routing policies (latency, failover, weighted routing)
- Production-grade scaling

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
DNS	Domain Name System
EC2	Elastic Compute Cloud
AWS	Amazon Web Services

NS	Name Server
A Record	Maps domain to IP address
CNAME	Canonical Name (alias)
BIND9	DNS server software
Route 53	AWS managed DNS service
TTL	Time To Live
FQDN	Fully Qualified Domain Name

1.4 References

- AWS Route 53 Documentation
- BIND9 Official Documentation
- Linux Manual Pages (dig, nslookup)
- AWS EC2 Documentation
- DNS RFC Standards (RFC 1034, RFC 1035)

1.5 Overview

This document covers:

- Project objectives and business goals
- Functional and non-functional requirements
- Use cases and system interactions
- Risks, assumptions, and dependencies
- Technical understanding of DNS systems

---

2. Project Overview

2.1 Project Objectives

- Deploy a self-hosted DNS server on EC2
- Configure a Route 53 hosted zone
- Create identical DNS records in both systems
- Switch NS records between both systems

- Analyze DNS resolution behavior
- Understand DNS propagation

2.2 Business Goals

- Evaluate cost vs performance of DNS solutions
- Understand control vs convenience trade-offs
- Improve internal knowledge of DNS infrastructure
- Enable better decision-making for DNS architecture

2.3 Background

DNS is a critical component of internet infrastructure. Organizations can either:

- Host their own DNS servers (self-managed)
- Use managed DNS services like AWS Route 53

This project is designed to compare both approaches practically by switching NS records and observing resolution behavior.

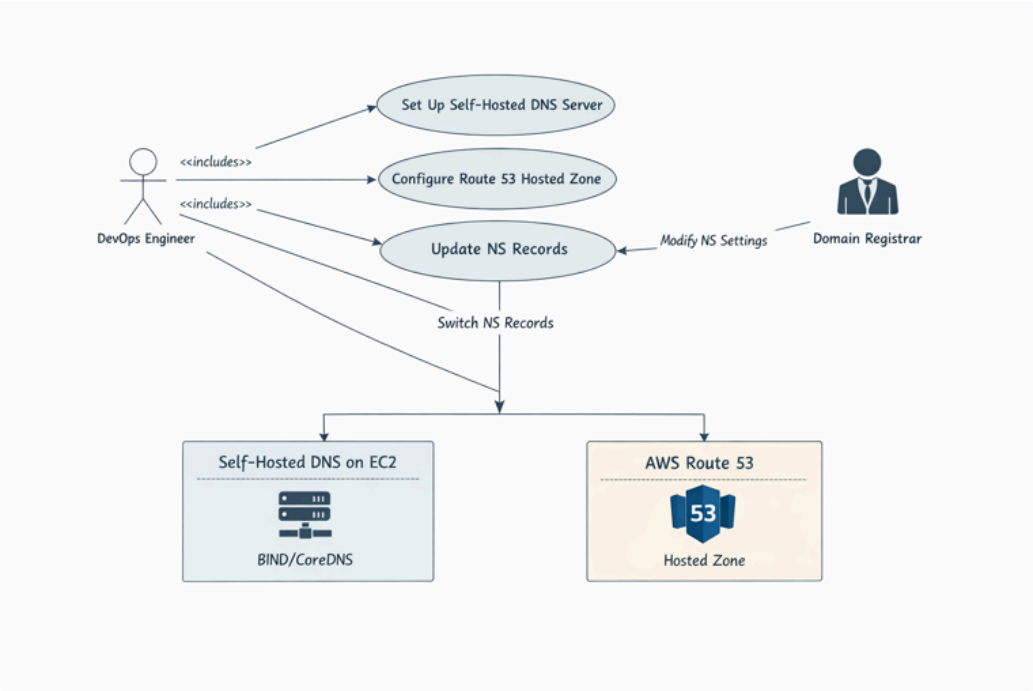
2.4 Constraints

- Time constraint: 3 hours
- Limited infrastructure (single EC2 instance)
- DNS propagation delay
- Dependency on domain registrar
- AWS service availability

---

3. Functional Requirements

3.1 Use Case Diagram



3.2 Use Cases

3.2.1 Use Case 1: Setup Self-Hosted DNS Server

Description:

Configure a DNS server on EC2 using BIND9 or CoreDNS.

Actors:

- DevOps Engineer

Preconditions:

- EC2 instance is running
- SSH access is available

Postconditions:

- DNS server is installed and running

Flow of Events:

1. Launch EC2 instance
2. Install BIND9/CoreDNS

3. Configure zone file
4. Start DNS service

Alternative Flows:

- Installation failure → Retry package installation
- Port 53 blocked → Update security group

3.2.2 Use Case 2: Configure Route 53 Hosted Zone

Description:
Create and configure DNS records in Route 53.

Actors:

- DevOps Engineer

Preconditions:

- AWS account is active

Postconditions:

- Hosted zone is created

Flow of Events:

1. Open Route 53
2. Create hosted zone
3. Add A and CNAME records
4. Save configuration

Alternative Flows:

- Incorrect domain name → Recreate hosted zone

3.2.3 Use Case 3: Update NS Records

Description:
Switch domain nameservers between EC2 DNS and Route 53.

Actors:

- DevOps Engineer
- Domain Registrar

Preconditions:

- DNS servers configured

Postconditions:

- Domain points to selected DNS system

Flow of Events:

1. Access domain registrar
2. Update NS records
3. Save changes
4. Wait for propagation

Alternative Flows:

- Wrong NS entry → DNS failure

3.2.4 Use Case 4: Test DNS Resolution

Description:
Verify DNS resolution using CLI tools.

Actors:

- DevOps Engineer

Preconditions:

- DNS configured

Postconditions:

- DNS response verified

Flow of Events:

1. Run dig/nslookup
2. Query domain
3. Analyze response
4. Identify authoritative server

Alternative Flows:

- No response → Check DNS server status

---

4. Non-Functional Requirements

4.1 Performance Requirements

- DNS query response time should be minimal (<200 ms typical)

4.2 Security Requirements

- Allow port 53 (TCP/UDP)
- Restrict access via security groups
- Prevent unauthorized DNS modifications

4.3 Usability Requirements

- Easy configuration via AWS Console
- Simple CLI commands for testing
- Clear DNS record structure

4.4 Reliability Requirements

- Route 53 → High availability
- Self-hosted → Depends on EC2 uptime

4.5 Compliance Requirements

- Follow standard DNS protocols (RFC 1034, 1035)
- Ensure no misuse of domain or DNS records

---

5. Assumptions and Dependencies

5.1 Assumptions

- Domain is already purchased
- AWS account is configured
- Internet connectivity is stable
- User has basic Linux knowledge

5.2 Dependencies

- AWS EC2 service
- AWS Route 53
- Domain registrar
- Network connectivity

---

6. Risk Analysis

6.1 Risk Identification

- Incorrect NS configuration
- DNS propagation delays
- EC2 downtime
- Misconfigured security groups

6.2 Risk Mitigation

- Validate NS records before saving
- Use dig +trace for debugging
- Monitor EC2 uptime
- Open required ports properly

---

7. Glossary

Term	Meaning
DNS	Resolves domain to IP
NS	Authoritative name server
Route 53	AWS DNS service
BIND9	DNS software
TTL	Cache duration

---

8. Appendices

- Sample DNS records
- dig command outputs
- Configuration files

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____